

Tarea 03 — Reglas de asociación y clasificación

Este documento presenta:

- Reglas de asociación (Apriori, FP-Growth)
- Clasificación

Contenido

- [Apriori](#)
- [Reglas de asociación](#)
- [Clasificación](#)

Implementación de Apriori propio

Funciones auxiliares

Las siguientes funciones de soporte calculan las métricas fundamentales utilizadas por el algoritmo Apriori propio.

```
from itertools import combinations

def get_support(itemset, transactions, n):
    # Soporte = proporción de transacciones que contienen el itemset
    count = sum(1 for t in transactions if itemset.issubset(t))
    return count / n

def get_confidence(sup_itemset, sup_ant):
    # Confianza = soporte del conjunto completo / soporte del antecedente
    return sup_itemset / sup_ant if sup_ant > 0 else 0

def get_lift(confidence, sup_cons):
    # Lift = qué tanto mejora la regla respecto a la casualidad
    return confidence / sup_cons if sup_cons > 0 else 0
```

Implementamos el algoritmo apriori propio utilizando las funciones auxiliares anteriores

```
from itertools import combinations

def get_support(itemset, transactions, n):
    count = sum(1 for t in transactions if itemset.issubset(t))
    return count / n

def get_confidence(sup_itemset, sup_ant):
    return sup_itemset / sup_ant if sup_ant > 0 else 0
```

```

def get_lift(confidence, sup_cons):
    return confidence / sup_cons if sup_cons > 0 else 0

def apriori_propio(transactions, min_support, min_confidence):
    """
    Implementación propia del algoritmo Apriori.
    Retorna (frequent_itemsets_dict, rules_list)
    """

    n = len(transactions)

    # extraemos todos los items únicos del dataset
    all_items = set(item for t in transactions for item in t)

    # itemsets frecuentes de tamaño 1
    freq = {}
    L1 = {}

    for item in all_items:
        # calculamos el soporte de cada item individual
        sup = get_support(frozenset([item]), transactions, n)
        # dejamos solo los que cumplan con el soporte mínimo
        if sup >= min_support:
            L1[frozenset([item])] = sup
    freq.update(L1)

    Lk = L1
    k = 2
    while Lk:
        prev_keys = list(Lk.keys()) # itemsets frecuentes del nivel anterior
        candidates = set()

        for i in range(len(prev_keys)):
            for j in range(i + 1, len(prev_keys)):
                union = prev_keys[i] | prev_keys[j]
                # consideramos solo los itemset de tamaño k
                if len(union) == k:
                    candidates.add(union)

        Lk_new = {}

        for c in candidates:

```

```

# poda: verificamos que todos los subconjuntos sean frecuentes
subsets_freq = all(
    frozenset(sub) in Lk
    for sub in combinations(c, k - 1)
)
if not subsets_freq:
    continue

# calculamos soporte del candidato
sup = get_support(c, transactions, n)
# cumple con soporte minimo
if sup >= min_support:
    Lk_new[c] = sup

freq.update(Lk_new)
Lk = Lk_new
k += 1

# Generar reglas
rules = []
for itemset, sup_itemset in freq.items():
    # solo los itemsets de tamaño >= 2 generan reglas
    if len(itemset) < 2:
        continue

    # generar todas las posibles particiones antecedente/consecuente
    for size in range(1, len(itemset)):
        for antecedent in combinations(itemset, size):
            antecedent = frozenset(antecedent)
            consequent = itemset - antecedent
            # soporte del antecedente
            sup_ant = freq.get(antecedent, get_support(antecedent, transactions, n))
            if sup_ant == 0:
                continue
            # calculamos confianza
            confidence = get_confidence(sup_itemset, sup_ant)
            # soporte del consecuente
            sup_cons = freq.get(consequent, get_support(consequent, transactions, n))
            # calculamos lift
            lift = get_lift(confidence, sup_cons)
            # filtrar por confianza mínima
            if confidence >= min_confidence:

```

```
        rules.append({
            "antecedents": antecedent,
            "consequents": consequent,
            "support":     round(sup_itemset, 4),
            "confidence":  round(confidence, 4),
            "lift":        round(lift, 4)
        })
    return freq, rules
```

Descubrimiento de patrones

Reglas de asociación

Descubrir patrones del tipo:

- SEXO = Femenino y GRUPO_EDAD = 12-17 $\rightarrow$ ENTIDAD = ...
- ENTIDAD = ... y MES_DESAPARICION = ... $\rightarrow$ ESTATUS_VICTIMA = ...

Se aplican los 3 algoritmos: Apriori propio (implementado en el capítulo anterior), mlxtend Apriori y FP-Growth, justificando cuál tarda más, cuál tarda menos y si generan las mismas reglas.

Preparación del dataset

Antes de aplicar los algoritmos de asociación, se preparó el dataset resolviendo tres aspectos clave: el manejo de datos CONFIDENCIAL, la discretización de variables continuas y la selección de columnas para las transacciones.

Manejo de datos confidenciales

Se eliminaron las filas que contienen algún dato confidencial en las variables clave, puesto que no aportan información semántica útil para descubrir patrones o construir modelos analíticos. Mantener estos valores como categoría podría producir reglas triviales, por ejemplo: “*confidencial* $\rightarrow$ *confidencial*”, sin valor interpretativo real.

Discretización de variables continuas

Apriori requiere ítems categóricos, por lo que se obtuvieron las siguientes categorías principales:

Table 1: Discretización de variables

Variable	Tipo	Transformación	Justificación
Edad	Numérica	0-11, 12-17, 18-29, 30-59, 60+	Etapas con diferente perfil de riesgo. Se calculó obteniendo la diferencia entre la fecha de desaparición y la fecha de nacimiento, expresada en días y convertida a años completos mediante división entera entre 365.
Mes de desaparición	Numérica (1-12)	Nombre del mes	Para tener mejor interpretación y legibilidad en el análisis.
Entidad	Categórica	Sin cambios	Ya es categórica con 32 estados.
Sexo	Categórica	Sin cambios	Variable categórica importante para el análisis.
Estatus de la víctima	Categórica	Sin cambios	Variable clave incluida en las reglas de asociación.
Año de desaparición	Numérica	Sexenio presidencial: Fox (2000-2006), Calderón (2006-2012), Peña Nieto (2012-2018), AMLO (2018-2024), Sheinbaum (2024-)	Permite analizar cambios en patrones según el sexenio. Se extrajo el año de la fecha de desaparición y se clasificó en los rangos correspondientes.

Se discretizaron las variables numéricas y se incluyeron variables categóricas existentes como sexo, entidad y estatus de la víctima para construir los ítems del algoritmo.

Implementación de los algoritmos

Implementación de mlxtend Apriori y FP-Growth

```
import time
from mlxtend.frequent_patterns import apriori as mlx_apriori
from mlxtend.frequent_patterns import fpgrowth
from mlxtend.frequent_patterns import association_rules

def apriori_mlxtend(df_te, min_support, min_confidence):
    """
    Apriori de mlxtend.
    Retorna (freq_itemsets, rules_df, elapsed_seconds)
    """
```

```

"""
t0 = time.time()
freq = mlx_apriori(df_te, min_support, use_colnames=True)
rules = association_rules(freq, metric="confidence", min_threshold=min_confidence)
elapsed = time.time() - t0
return freq, rules, elapsed

```

```

import time
from mlxtend.frequent_patterns import apriori as mlx_apriori
from mlxtend.frequent_patterns import fpgrowth
from mlxtend.frequent_patterns import association_rules

def apriori_fpgrowth(df_te, min_support, min_confidence):
    """
    FP-Growth de mlxtend.
    Retorna (freq_itemsets, rules_df, elapsed_seconds)
    """
    t0 = time.time()
    freq = fpgrowth(df_te, min_support, use_colnames=True)
    rules = association_rules(freq, metric="confidence", min_threshold=min_confidence)
    elapsed = time.time() - t0
    return freq, rules, elapsed

```

Se siguieron los siguientes pasos:

Paso 1 — Codificación de transacciones

```

import pandas as pd

def encode_transactions(df_enc, columns):
    """
    Transforma los valores del dataset a la forma {col}={val},
    como SEXO=MUJER. Lo utiliza Apriori propio.
    """
    transactions = []
    for _, row in df_enc.iterrows():
        items = set()
        for col in columns:
            val = row[col]

```



```

        if pd.notna(val) and str(val).strip() not in ("", "nan"):
            items.add(f"{col}={val}")
        transactions.append(frozenset(items))
    return transactions

```

Esta función cambia el formato de cada registro del dataset en una transacción, representándola como un conjunto de ítems. Cada ítem se construye concatenando el nombre de la variable con su valor (por ejemplo, SEXO=MUJER), lo que permite generar representaciones atómicas e interpretables para el algoritmo. Esto es necesario porque Apriori trabaja con conjuntos de ítems categóricos.

```

from mlxtend.preprocessing import TransactionEncoder

def transactions_to_df(transactions):
    """
    Convierte lista de frozensets al DataFrame booleano
    que requiere mlxtend y FP-Growth.
    """
    te = TransactionEncoder()
    te_array = te.fit_transform([list(t) for t in transactions])
    return pd.DataFrame(te_array, columns=te.columns_)

```

Esta función transforma el formato del dataset a un conjunto de columnas con filas booleanas, útil para los otros dos algoritmos implementados.

Paso 2 — Búsqueda de ítems frecuentes

Los 3 algoritmos reciben las transacciones con el formato específico y buscan combinaciones de ítems (*itemsets*) que aparezcan en al menos el 4% de las transacciones (`min_support = 0.04`).

Un **itemset frecuente** es cualquier combinación de categorías que supera ese umbral mínimo. Por ejemplo:

- {SEXO=Masculino} → 1-itemset frecuente si aparece en $\geq 4\%$ de las transacciones
- {SEXO=Femenino, GRUPO_EDAD=12-17} → 2-itemset frecuente si ocurren en $\geq 4\%$
- {SEXO=Femenino, GRUPO_EDAD=12-17, ESTATUS=No Localizado} → 3-itemset

Solo los itemsets que superan el soporte mínimo se mantienen. Esta fase produce el conjunto de todos los itemsets frecuentes, que en el paso siguiente se particionan en antecedente y consecuente para generar las reglas de asociación.

- **Apriori propio:** genera combinaciones nivel a nivel (1-itemsets $\rightarrow$ 2-itemsets $\rightarrow$ 3-itemsets...), quitando los que no cumplen el soporte mínimo. Es el algoritmo más lento porque escanea todas las transacciones en cada nivel.
- **mlxtend Apriori:** tiene la misma lógica, pero su implementación está optimizada en C.
- **FP-Growth:** construye un árbol compacto (FP-Tree) del dataset y extrae patrones directamente del árbol sin enumerar combinaciones posibles. Es el más rápido por la estructura de datos que utiliza.

Sin embargo, los tres llegan a los mismos itemsets frecuentes; la diferencia es solo la velocidad.

Paso 3 — Generación de reglas y métricas

De cada itemset frecuente de tamaño ≥ 2 , se generan todas las particiones posibles de la forma antecedente $\rightarrow$ consecuente y se conservan las que cumplan `min_confidence = 0.04`.

Table 2: Métricas de asociación

Métrica	Fórmula	Explicación
Support	$s(A \rightarrow B) = \frac{ \{T \in D \mid A \cup B \subseteq T\} }{ D }$	Qué tan frecuente es el patrón en el dataset.
Confidence	$c(A \rightarrow B) = \frac{ \{T \in D \mid A \cup B \subseteq T\} }{ \{T \in D \mid A \subseteq T\} }$	Probabilidad de que ocurra B dado A .
Lift	$\text{lift}(X \Rightarrow Y) = \frac{\text{soporte}(X \cup Y)}{\text{soporte}(X) \cdot \text{soporte}(Y)}$	Fuerza de la asociación entre antecedente y consecuente.

```
def get_support(itemset, transactions, n):
    count = sum(1 for t in transactions if itemset.issubset(t))
    return count / n

def get_confidence(A, B, transactions):
    support_AB = sum(1 for t in transactions if A.union(B).issubset(t))
    support_A = sum(1 for t in transactions if A.issubset(t))
    return support_AB / support_A

def get_lift(confidence, sup_cons):
    return confidence / sup_cons if sup_cons > 0 else 0
```

Las funciones anteriores son utilizadas únicamente por **Apriori propio**. Los algoritmos de mlxtend calculan estas métricas internamente con `mlx_apriori`, `fpgrowth` y `association_rules`.

Paso 4 — Filtración de reglas

Se aplica un filtro adicional con `lift > 1.2` y `confidence > 0.4` para mantener únicamente patrones con una relación estadísticamente fuerte.

```
# Reglas interesantes (lift > 1.2, conf > 0.4)
if len(rules_fp) > 0:
    mask = (rules_fp["lift"] > 1.2) & (rules_fp["confidence"] > 0.4)
    interesting = rules_fp[mask].sort_values("lift", ascending=False)
    print(f"\n    Reglas interesantes (lift>1.2, conf>0.4): {len(interesting)}    ")
    for _, r in interesting.head(5).iterrows():
        ant = ", ".join(sorted(r["antecedents"]))
        con = ", ".join(sorted(r["consequents"]))
        print(f"    [{ant}] → [{con}]")
        print(f"    sup={r['support']:.3f}    conf={r['confidence']:.3f}    lift={r['lift']:.3f}")
```

Resultados por sexenio

Para cada sexenio se reportan: el número de transacciones utilizadas, los tiempos de ejecución de cada algoritmo, el número de itemsets frecuentes y reglas generadas, las cinco reglas con mayor lift, y las reglas bajo el criterio `lift > 1.2` y `confianza > 0.4`.

Tabla resumen de tiempos

Table 3: Tiempos de ejecución y número de reglas por sexenio

Sexenio	Transacciones	t_propio (s)	t_mlxtend		Reglas
			(s)	t_fp (s)	
Fox (2000–2006)	600	0.03	0.01	0.01	197
Calderón (2006–2012)	7 039	0.40	0.02	0.04	239
Peña Nieto (2012–2018)	11 705	0.65	0.03	0.06	198
AMLO (2018–2024)	24 632	1.56	0.05	0.11	174
Sheinbaum (2024–)	6 476	0.37	0.01	0.03	232

Los tres algoritmos producen exactamente el mismo número de reglas en cada sexenio. Esto confirma que **Apriori propio, mlxtend Apriori y FP-Growth son matemáticamente equivalentes**, ya que parten del mismo umbral de soporte mínimo y recorren el mismo espacio de itemsets frecuentes. La diferencia radica únicamente en la estrategia de búsqueda y en la eficiencia de implementación.

Fox (2000–2006) — 600 transacciones

Con 600 registros válidos (el sexenio con menos registros en el dataset) se encontraron **22 reglas de asociación** con $\text{lift} > 1.2$ y $\text{conf} > 0.4$.

La regla de mayor lift en este período es:

MES_DESAPARICION = Mayo $\rightarrow$ GRUPO_EDAD = 18-29, ESTATUS_VICTIMA = DESAPARECIDA
support = 0.045, confidence = 0.50, lift = 1.622

Durante el sexenio de Fox, las desapariciones registradas en mayo tenían un 62% más de probabilidad de involucrar a jóvenes de 18 a 29 años aún no localizados, en comparación con lo esperado por azar.

Otra regla destacable es:

ENTIDAD=ESTADO DE MÉXICO $\rightarrow$ SEXO=MUJER, ESTATUS_VICTIMA=DESAPARECIDA lift = 1.52

Apunta al Estado de México como entidad con mayor proporción de mujeres desaparecidas no localizadas durante ese período.

Calderón (2006–2012) — 7 039 transacciones

En el sexenio de Calderón, el número de registros válidos aumenta considerablemente. Sin embargo, el número de reglas de asociación se reduce a **2**, lo que indica que los patrones son más dispersos y no se encuentra ninguna combinación de variables con suficiente fuerza estadística.

Las dos reglas que superan los umbrales son:

$$\{SEXO = MUJER\} \rightarrow \{GRUPO_EDAD = 18-29, ESTATUS_VICTIMA = DESAPARECIDA\} \quad \text{lift}$$

$$\{ENTIDAD = TAMAULIPAS\} \rightarrow \{GRUPO_EDAD = 18-29, ESTATUS_VICTIMA = DESAPARECIDA\} \quad \text{lift}$$

Tamaulipas fue entidad clave del sexenio de Calderón, con confianza de 0.464 y soporte de 0.111 —el soporte más alto de todas las reglas de asociación en cualquier sexenio. El perfil predominante es **Mujer, 18-29 años, Tamaulipas, Desaparecida**.

Peña Nieto (2012–2018) — 11 705 transacciones

Este sexenio produce las reglas con mayor lift de todo el análisis. Se encontraron **13 reglas de asociación**. La regla principal es:

$$\{GRUPO_EDAD = 12-17\} \rightarrow \{SEXO = MUJER, ESTATUS_VICTIMA = DESAPARECIDA\} \quad \text{lift}$$

Con un lift de 3.13, la probabilidad de que una víctima de 12 a 17 años sea mujer y no localizada es más del triple de lo que se esperaría si sexo, edad y estatus fueran independientes. Esta regla es estadísticamente robusta (conf = 0.602, sup = 0.051, equivalente a 596 casos reales sobre 11 705).

Otra regla destacable es que el Estado de México concentra el patrón de adolescentes desaparecidos:

$$\{GRUPO_EDAD = 12-17\} \rightarrow \{ENTIDAD = ESTADO DE MEXICO, ESTATUS_VICTIMA = DESAPARECIDA\} \quad \text{lift}$$

Estas reglas no reflejan que el fenómeno sea exclusivo del **Estado de México**, sino que concentra el mayor número de registros en esta categoría.

AMLO (2018–2024) — 24 632 transacciones

Fue el sexenio con la mayor cantidad de transacciones, lo que permite deducir que fue el período con más desapariciones. Es donde más tiempo tardó el algoritmo de Apriori propio (1.56 s), confirmando el comportamiento $O(n)$ en el número de transacciones, a diferencia de FP-Growth y mlxtend que se mantuvieron por debajo de 0.12 s.

Se encontraron **2 reglas de asociación**:

$$\{GRUPO_EDAD = 12-17\} \rightarrow \{SEXO = MUJER\} \quad \text{lift} = 2.562$$

$$\{GRUPO_EDAD = 12-17\} \rightarrow \{SEXO = MUJER, ESTATUS_VICTIMA = DESAPARECIDA\} \quad \text{lift} = 1.747$$

El patrón de **niñas y adolescentes (12–17 años) desaparecidas y no localizadas** se mantiene pero con un lift menor que en el período anterior (2.5), lo que sugiere que las desapariciones se volvieron más generalizadas y distribuidas geográficamente, ya no concentradas explícitamente en el Estado de México (aunque éste conserva un lift de 1.747 para mujeres desaparecidas).

Sheinbaum (2024–presente) — 6 476 transacciones

Este sexenio es el actual; los datos llegan hasta septiembre de 2025. A pesar de eso, se encontraron **12 reglas de asociación**, el segundo mayor número después del período de Fox.

La regla más fuerte sigue siendo el patrón de adolescentes mujeres:

$$\{GRUPO_EDAD = 12-17\} \rightarrow \{SEXO = MUJER, ESTATUS_VICTIMA = DESAPARECIDA\} \quad \text{lift} = 2.562$$

Aparece además una regla nueva que no existía en sexenios anteriores:

$$\{ENTIDAD = SINALOA\} \rightarrow \{GRUPO_EDAD = 30-59, SEXO = HOMBRE, ESTATUS_VICTIMA = DESAPARECIDA\} \quad \text{lift} = 1.747$$

Lo que apunta a un desplazamiento del perfil de víctimas en Sinaloa hacia hombres adultos en este nuevo período.

Comparativa entre los tres algoritmos

¿Generan las mismas reglas?

Sí, en todos los sexenios los tres algoritmos producen exactamente el mismo conjunto de itemsets frecuentes y el mismo número de reglas. Esto se debe a que los tres resuelven el mismo problema matemático (encontrar todos los itemsets cuyo soporte supere el umbral mínimo) y usan la misma métrica de confianza para generar reglas. La equivalencia se garantiza siempre que se use el mismo `min_support` y `min_confidence`.

¿Cuál tarda más?

El algoritmo **Apriori propio** es el más lento en todos los sexenios, puesto que genera combinaciones de tamaño k enumerando todos los pares de itemsets frecuentes de tamaño $k - 1$, y luego escanea todas las transacciones para calcular el soporte de cada candidato. Al estar implementado en Python puro, sin vectorización, cada operación de subconjunto recorre listas con bucles nativos y su tiempo crece de forma aproximadamente lineal con el número de transacciones.

¿Cuál tarda menos?

El algoritmo **mlxtend Apriori** es el más rápido en los sexenios de menor tamaño, mientras que **FP-Growth** es ligeramente más lento que **mlxtend** en este dataset en específico, pero escalaría mejor en datasets mucho más grandes, ya que no enumera combinaciones: comprime el dataset en un árbol FP-Tree y extrae patrones frecuentes recorriéndolo, eliminando el cuello de botella de generar y podar combinaciones.

Interpretación general de los patrones

Del análisis por sexenio emergen tres patrones transversales:

1. **Desaparición de niñas y adolescentes (12–17 años):** Es el patrón más fuerte y consistente a lo largo del tiempo, presente desde Peña Nieto hasta Sheinbaum con lifts entre 2.2 y 3.1. El Estado de México, Sonora y Tamaulipas concentran la mayor parte de estos casos durante 2012–2024.
2. **Desaparición de hombres en edad de 30–59 años:** Durante el sexenio de Calderón, Tamaulipas destaca como la entidad con mayor asociación con desapariciones de hombres de 18–29 años, reflejando el período de mayor intensidad del conflicto entre organizaciones criminales en esa zona. En el período de Sheinbaum, Sinaloa toma un rol similar para hombres adultos (30–59 años).
3. **Casos frecuentes durante Gobierno de Fox:** Durante el sexenio de Fox, el mes de mayo aparece asociado a desapariciones de jóvenes, aunque con soporte muy bajo. Este patrón no se reproduce en sexenios posteriores.

Limitaciones y posibles sesgos

- **Sesgo de registro:** El 37% de los registros originales son CONFIDENCIAL y fueron excluidos del análisis. Si la confidencialidad no es aleatoria (por ejemplo, si las fiscalías de ciertos estados clasifican más casos como confidenciales), las reglas pueden estar sesgadas hacia las entidades con mayor transparencia en el registro.
- **Dominio de estatus DESAPARECIDA:** El estatus de desaparecido representa el 94.6% de los registros válidos. Las reglas que incluyen este consecuente tienen soporte y confianza altos simplemente porque es el valor mayoritario, no necesariamente porque exista una asociación real. El lift corrige parcialmente esto, por lo que las reglas con $\text{lift} > 1.2$ son las que realmente aportan información.
- **Cobertura desigual por sexenio:** Fox tiene apenas 600 registros mientras que AMLO tiene 24 632. Hay diferentes razones: en el sexenio de Fox no existía la tecnología actual y podrían haberse perdido registros, o realmente hubo mayor número de desapariciones durante el sexenio de AMLO. Por lo tanto, las comparaciones entre sexenios deben hacerse con reservas.

Código completo:

```
import pandas as pd
import numpy as np
import time
import warnings
from itertools import combinations
from collections import defaultdict
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori as mlx_apriori
from mlxtend.frequent_patterns import fpgrowth, association_rules

warnings.filterwarnings("ignore")

#
# Algoritmos implementados
#

def apriori_propio(transactions, min_support, min_confidence):
    """
    Implementación propia del algoritmo Apriori.
```



```

Retorna (frequent_itemsets_dict, rules_list)
"""
n = len(transactions)
all_items = set(item for t in transactions for item in t)

# Itemsets frecuentes de tamaño 1
freq = {}
L1 = {}
for item in all_items:
    sup = get_support(frozenset([item]), transactions, n)
    if sup >= min_support:
        L1[frozenset([item])] = sup
freq.update(L1)

Lk = L1
k = 2
while Lk:
    prev_keys = list(Lk.keys())
    candidates = set()
    for i in range(len(prev_keys)):
        for j in range(i + 1, len(prev_keys)):
            union = prev_keys[i] | prev_keys[j]
            if len(union) == k:
                candidates.add(union)

    Lk_new = {}
    for c in candidates:
        subsets_freq = all(
            frozenset(sub) in Lk
            for sub in combinations(c, k - 1)
        )
        if not subsets_freq:
            continue
        sup = get_support(c, transactions, n)
        if sup >= min_support:
            Lk_new[c] = sup

    freq.update(Lk_new)
    Lk = Lk_new
    k += 1

# Generar reglas

```

```

rules = []
for itemset, sup_itemset in freq.items():
    if len(itemset) < 2:
        continue
    for size in range(1, len(itemset)):
        for antecedent in combinations(itemset, size):
            antecedent = frozenset(antecedent)
            consequent = itemset - antecedent
            sup_ant = freq.get(antecedent, get_support(antecedent, transactions, n))
            if sup_ant == 0:
                continue
            confidence = get_confidence(sup_itemset, sup_ant) # calculamos confidence
            sup_cons = freq.get(consequent, get_support(consequent, transactions, n))
            lift = get_lift(confidence, sup_cons) # calculamos lift
            if confidence >= min_confidence:
                rules.append({
                    "antecedents": antecedent,
                    "consequents": consequent,
                    "support": round(sup_itemset, 4),
                    "confidence": round(confidence, 4),
                    "lift": round(lift, 4)
                })
return freq, rules

def apriori_mlxtend(df_te, min_support, min_confidence):
    """
    Apriori de mlxtend.
    Retorna (freq_itemsets, rules_df, elapsed_seconds)
    """
    t0 = time.time()
    freq = mlx_apriori(df_te, min_support, use_colnames=True)
    rules = association_rules(freq, metric="confidence", min_threshold=min_confidence)
    elapsed = time.time() - t0
    return freq, rules, elapsed

def apriori_fpgrowth(df_te, min_support, min_confidence):
    """
    FP-Growth de mlxtend.
    Retorna (freq_itemsets, rules_df, elapsed_seconds)
    """

```

```

t0 = time.time()
freq = fpgrowth(df_te, min_support, use_colnames=True)
rules = association_rules(freq, metric="confidence", min_threshold=min_confidence)
elapsed = time.time() - t0
return freq, rules, elapsed

#
# Codificación de transacciones
#
def encode_transactions(df_enc, columns):
    """Transforma los valores del dataset a la forma {col}={val}, como SEXO=mujer, lo utiliz
    transactions = []
    for _, row in df_enc.iterrows():
        items = set()
        for col in columns:
            val = row[col]
            if pd.notna(val) and str(val).strip() not in ("", "nan"):
                items.add(f"{col}={val}")
        transactions.append(frozenset(items))
    return transactions

def transactions_to_df(transactions):
    """Convierte lista de frozensets al DataFrame booleano que requiere mlxtend y fp-growth"
    te = TransactionEncoder()
    te_array = te.fit_transform([list(t) for t in transactions])
    return pd.DataFrame(te_array, columns=te.columns_)

#
# Calculamos Support, confidence, lift
#

def get_support(itemset, transactions, n):
    count = sum(1 for t in transactions if itemset.issubset(t))
    return count / n

def get_confidence(sup_itemset, sup_ant):
    return sup_itemset / sup_ant if sup_ant > 0 else 0

def get_lift(confidence, sup_cons):
    return confidence / sup_cons if sup_cons > 0 else 0

```

```

#
# 0. CARGA Y PREPROCESAMIENTO
#
print("=" * 65)
print("0. CARGA Y PREPROCESAMIENTO")
print("=" * 65)

df_raw = pd.read_csv("data_secretariado.csv")
print(f"Registros originales : {len(df_raw):,}")
print(f"Columnas           : {list(df_raw.columns)}\n")

CONF = "CONFIDENCIAL"

# Filtrar CONFIDENCIAL para reglas de asociación
df = df_raw[df_raw["SEXO"] != CONF].copy()
df = df[df["FECHA_DESAPARICION"] != CONF].copy()
df = df[df["FECHA_NACIMIENTO"] != CONF].copy()
df = df[df["ESTATUS_VICTIMA"] != CONF].copy()
print(f"Registros tras filtrar CONFIDENCIAL: {len(df):,}")

df["FECHA_NACIMIENTO"] = pd.to_datetime(df["FECHA_NACIMIENTO"], errors="coerce")
df["FECHA_DESAPARICION"] = pd.to_datetime(df["FECHA_DESAPARICION"], errors="coerce")
df.dropna(subset=["FECHA_NACIMIENTO", "FECHA_DESAPARICION"], inplace=True)

df["EDAD"] = (df["FECHA_DESAPARICION"] - df["FECHA_NACIMIENTO"]).dt.days // 365
df = df[(df["EDAD"] >= 0) & (df["EDAD"] <= 110)]
print(f"Registros tras filtrar edades inválidas: {len(df):,}\n")

bins_edad = [0, 11, 17, 29, 59, 110]
labs_edad = ["0-11", "12-17", "18-29", "30-59", "60+"]
df["GRUPO_EDAD"] = pd.cut(df["EDAD"], bins=bins_edad, labels=labs_edad, right=True)
df["MES_DESAPARICION"] = df["FECHA_DESAPARICION"].dt.month.map(
    {1:"Enero",2:"Febrero",3:"Marzo",4:"Abril",5:"Mayo",6:"Junio",
     7:"Julio",8:"Agosto",9:"Septiembre",10:"Octubre",11:"Noviembre",12:"Diciembre"}
)
df["SEXO"] = df["SEXO"].str.strip()
df.drop_duplicates(subset="ID_VICTIMA", inplace=True)
print(f"Registros finales para asociación: {len(df):,}\n")

for col in ["SEXO", "GRUPO_EDAD", "ESTATUS_VICTIMA"]:

```

```

    print(f" {col}:\n{df[col].value_counts().to_string()}\n")

# Segmentación temporal por sexenio
def asignar_sexenio(fecha):
    y = fecha.year
    if y <= 2006: return "Fox (2000-2006)"
    elif y <= 2012: return "Calderon (2006-2012)"
    elif y <= 2018: return "PeñaNieto (2012-2018)"
    elif y <= 2024: return "AMLO (2018-2024)"
    else: return "Sheinbaum (2024-)"

df["SEXENIO"] = df["FECHA_DESAPARICION"].apply(asignar_sexenio)
print("Distribución por sexenio:")
print(df["SEXENIO"].value_counts().sort_index().to_string(), "\n")

#
# 1 y 2. REGLAS DE ASOCIACIÓN - 3 algoritmos por sexenio
#
print("=" * 65)
print("REGLAS DE ASOCIACIÓN POR SEXENIO - Apriori propio, mlxtend Apriori y FP-Growth")
print("=" * 65)

MIN_SUPPORT = 0.04
MIN_CONFIDENCE = 0.4
cols_assoc = ["SEXO", "GRUPO_EDAD", "ENTIDAD", "MES_DESAPARICION", "ESTATUS_VICTIMA"]

ORDEN_SEXENIOS = [
    "Fox (2000-2006)",
    "Calderon (2006-2012)",
    "PeñaNieto (2012-2018)",
    "AMLO (2018-2024)",
    "Sheinbaum (2024-)",
]

def _print_top5(rules_df, label):
    print(f" Top-5 reglas ({label}) por lift:")
    top = rules_df.sort_values("lift", ascending=False).head(5)
    for _, r in top.iterrows():
        ant = ", ".join(sorted(r["antecedents"]))
        con = ", ".join(sorted(r["consequents"]))
        print(f" [{ant}]")

```

```

        print(f"        → [{con}]\n")
        print(f"        sup={r['support']:.3f}   conf={r['confidence']:.3f}   lift={r['lift']:.3f}\n")

resumen_tiempos = []

for sexenio in ORDEN_SEXENIOS:
    df_seg = df[df["SEXENIO"] == sexenio]
    if len(df_seg) < 500:
        print(f"\n  [Saltando {sexenio}: solo {len(df_seg)} registros]\n")
        continue

    print(f"\n{'=' * 65}")
    print(f"  SEXENIO: {sexenio}  ({len(df_seg):,} registros)")
    print(f"{'=' * 65}")

    df_assoc = df_seg[cols_assoc].dropna().copy()
    transactions = encode_transactions(df_assoc, cols_assoc)
    df_te = transactions_to_df(transactions)

    print(f"  Transacciones usadas: {len(transactions):,}")
    print(f"  Parámetros → min_support={MIN_SUPPORT}, min_confidence={MIN_CONFIDENCE}\n")

    # Apriori propio
    print("\n  [Apriori propio] Ejecutando...")
    t0 = time.time()
    freq_propio, rules_propio = apriori_propio(transactions, MIN_SUPPORT, MIN_CONFIDENCE)
    t_propio = time.time() - t0
    print(f"    Tiempo: {t_propio:.2f} s  |  Itemsets: {len(freq_propio)}  |  Reglas: {len(rules_propio)}")
    if rules_propio:
        rules_propio_df = pd.DataFrame(rules_propio).sort_values("lift", ascending=False)
        _print_top5(rules_propio_df, "Apriori propio")

    # mlxtend Apriori
    print("\n  [mlxtend Apriori] Ejecutando...")
    freq_mlx, rules_mlx, t_mlx = apriori_mlxtend(df_te, MIN_SUPPORT, MIN_CONFIDENCE)
    print(f"    Tiempo: {t_mlx:.2f} s  |  Itemsets: {len(freq_mlx)}  |  Reglas: {len(rules_mlx)}")
    if len(rules_mlx) > 0:
        _print_top5(rules_mlx, "mlxtend Apriori")

    # FP-Growth
    print("\n  [FP-Growth] Ejecutando...")
    freq_fp, rules_fp, t_fp = apriori_fpgrowth(df_te, MIN_SUPPORT, MIN_CONFIDENCE)

```

```

print(f"    Tiempo: {t_fp:.2f} s | Itemsets: {len(freq_fp)} | Reglas: {len(rules_fp)}")
if len(rules_fp) > 0:
    _print_top5(rules_fp, "FP-Growth")

# Reglas interesantes (lift > 1.2, conf > 0.4)
if len(rules_fp) > 0:
    mask = (rules_fp["lift"] > 1.2) & (rules_fp["confidence"] > 0.4)
    interesting = rules_fp[mask].sort_values("lift", ascending=False)
    print(f"\n    Reglas interesantes (lift>1.2, conf>0.4): {len(interesting)} ")
    for _, r in interesting.head(5).iterrows():
        ant = ", ".join(sorted(r["antecedents"]))
        con = ", ".join(sorted(r["consequents"]))
        print(f"        [{ant}] → [{con}]")
        print(f"        sup={r['support']:.3f}  conf={r['confidence']:.3f}  lift={r['lift']:.3f}")

print(f"\n    Comparativa de tiempos ({sexenio}) ")
print(f"    Apriori propio : {t_propio:.2f} s")
print(f"    mlxtend Apriori : {t_mlx:.2f} s")
print(f"    FP-Growth      : {t_fp:.2f} s")

resumen_tiempos.append({
    "sexenio": sexenio, "registros": len(df_assoc),
    "t_propio": t_propio, "t_mlx": t_mlx, "t_fp": t_fp,
    "reglas_propio": len(rules_propio),
    "reglas_mlx": len(rules_mlx), "reglas_fp": len(rules_fp),
})

print("\n" + "=" * 65)
print("RESUMEN COMPARATIVO DE TIEMPOS POR SEXENIO")
print("=" * 65)
resumen_df = pd.DataFrame(resumen_tiempos)
print(resumen_df.to_string(index=False))

# variable dummy para que el bloque de RESUMEN FINAL no falle
# (toma los tiempos del último sexenio procesado)
if resumen_tiempos:
    t_propio = resumen_tiempos[-1]["t_propio"]
    t_mlx = resumen_tiempos[-1]["t_mlx"]
    t_fp = resumen_tiempos[-1]["t_fp"]

print("\n" + "=" * 65)

```

```

print("RESUMEN FINAL")
print("=" * 65)
print(f"\nTiempos algoritmos Apriori:")
print(f"  Apriori propio   : {t_propio:.2f} s  (O(2^n) candidatos, el mas lento)")
print(f"  mlxtend Apriori   : {t_mlx:.2f} s")
print(f"  FP-Growth         : {t_fp:.2f} s  (sin generación de candidatos, el más rápido)")

```

O. CARGA Y PREPROCESAMIENTO

Registros originales : 133,887

Columnas : ['ID_VICTIMA', 'ORIGEN_REPORTE', 'FECHA_NACIMIENTO', 'SEXO', 'FECHA_D

Registros tras filtrar CONFIDENCIAL: 84,738

Registros tras filtrar edades inválidas: 50,651

Registros finales para asociación: 50,651

SEXO:

SEXO

HOMBRE 40178

MUJER 10418

INDETERMINADO 55

GRUPO_EDAD:

GRUPO_EDAD

30-59 23389

18-29 18291

12-17 4473

60+ 2406

0-11 1893

ESTATUS_VICTIMA:

ESTATUS_VICTIMA

DESAPARECIDA 47899

NO LOCALIZADA 2752

Distribución por sexenio:

SEXENIO

AMLO (2018-2024) 24729

Calderon (2006-2012) 7043

Fox (2000-2006) 613

PeñaNieto (2012-2018) 11751
Sheinbaum (2024-) 6515

=====

REGLAS DE ASOCIACIÓN POR SEXENIO - Apriori propio, mlxtend Apriori y FP-Growth

=====

=====

SEXENIO: Fox (2000-2006) (613 registros)

=====

Transacciones usadas: 600
Parámetros → min_support=0.04, min_confidence=0.4

[Apriori propio] Ejecutando...

Tiempo: 0.03 s | Itemsets: 126 | Reglas: 185

Top-5 reglas (Apriori propio) por lift:

[MES_DESAPARICION=Mayo]

→ [ESTATUS_VICTIMA=DESAPARECIDA, GRUPO_EDAD=18-29]
sup=0.045 conf=0.500 lift=1.622

[ESTATUS_VICTIMA=DESAPARECIDA, MES_DESAPARICION=Mayo]

→ [GRUPO_EDAD=18-29]
sup=0.045 conf=0.509 lift=1.567

[MES_DESAPARICION=Mayo]

→ [GRUPO_EDAD=18-29]
sup=0.045 conf=0.500 lift=1.538

[ENTIDAD=SONORA]

→ [ESTATUS_VICTIMA=DESAPARECIDA, SEXO=HOMBRE]
sup=0.042 conf=0.962 lift=1.276

[MES_DESAPARICION=Enero, SEXO=HOMBRE]

→ [GRUPO_EDAD=30-59]
sup=0.065 conf=0.600 lift=1.263

[mlxtend Apriori] Ejecutando...

Tiempo: 0.01 s | Itemsets: 126 | Reglas: 185

Top-5 reglas (mlxtend Apriori) por lift:

[MES_DESAPARICION=Mayo]

→ [ESTATUS_VICTIMA=DESAPARECIDA, GRUPO_EDAD=18-29]
sup=0.045 conf=0.500 lift=1.622

[ESTATUS_VICTIMA=DESAPARECIDA, MES_DESAPARICION=Mayo]

→ [GRUPO_EDAD=18-29]
sup=0.045 conf=0.509 lift=1.567

[MES_DESAPARICION=Mayo]

```

→ [GRUPO_EDAD=18-29]
  sup=0.045  conf=0.500  lift=1.538
[ENTIDAD=SONORA]
→ [ESTATUS_VICTIMA=DESAPARECIDA, SEXO=HOMBRE]
  sup=0.042  conf=0.962  lift=1.276
[ESTATUS_VICTIMA=DESAPARECIDA, MES_DESAPARICION=Enero, SEXO=HOMBRE]
→ [GRUPO_EDAD=30-59]
  sup=0.060  conf=0.600  lift=1.263

[FP-Growth] Ejecutando...
  Tiempo: 0.01 s | Itemsets: 126 | Reglas: 185
Top-5 reglas (FP-Growth) por lift:
[MES_DESAPARICION=Mayo]
→ [ESTATUS_VICTIMA=DESAPARECIDA, GRUPO_EDAD=18-29]
  sup=0.045  conf=0.500  lift=1.622
[ESTATUS_VICTIMA=DESAPARECIDA, MES_DESAPARICION=Mayo]
→ [GRUPO_EDAD=18-29]
  sup=0.045  conf=0.509  lift=1.567
[MES_DESAPARICION=Mayo]
→ [GRUPO_EDAD=18-29]
  sup=0.045  conf=0.500  lift=1.538
[ENTIDAD=SONORA]
→ [ESTATUS_VICTIMA=DESAPARECIDA, SEXO=HOMBRE]
  sup=0.042  conf=0.962  lift=1.276
[ESTATUS_VICTIMA=DESAPARECIDA, MES_DESAPARICION=Enero, SEXO=HOMBRE]
→ [GRUPO_EDAD=30-59]
  sup=0.060  conf=0.600  lift=1.263

Reglas interesantes (lift>1.2, conf>0.4): 22
[MES_DESAPARICION=Mayo] → [ESTATUS_VICTIMA=DESAPARECIDA, GRUPO_EDAD=18-
29]
  sup=0.045  conf=0.500  lift=1.622
[ESTATUS_VICTIMA=DESAPARECIDA, MES_DESAPARICION=Mayo] → [GRUPO_EDAD=18-
29]
  sup=0.045  conf=0.509  lift=1.567
[MES_DESAPARICION=Mayo] → [GRUPO_EDAD=18-29]
  sup=0.045  conf=0.500  lift=1.538
[ENTIDAD=SONORA] → [ESTATUS_VICTIMA=DESAPARECIDA, SEXO=HOMBRE]
  sup=0.042  conf=0.962  lift=1.276
[ESTATUS_VICTIMA=DESAPARECIDA, MES_DESAPARICION=Enero, SEXO=HOMBRE] → [GRUPO_EDAD=30-
59]
  sup=0.060  conf=0.600  lift=1.263

```

Comparativa de tiempos (Fox (2000-2006))
Apriori propio : 0.03 s
mlxtend Apriori : 0.01 s
FP-Growth : 0.01 s

=====

SEXENIO: Calderon (2006-2012) (7,043 registros)

=====

Transacciones usadas: 7,039
Parámetros → min_support=0.04, min_confidence=0.4

[Apriori propio] Ejecutando...

Tiempo: 0.42 s | Itemsets: 140 | Reglas: 230
Top-5 reglas (Apriori propio) por lift:
[SEXO=MUJER]
→ [ESTATUS_VICTIMA=DESAPARECIDA, GRUPO_EDAD=18-29]
sup=0.071 conf=0.465 lift=1.204
[ENTIDAD=TAMAULIPAS]
→ [ESTATUS_VICTIMA=DESAPARECIDA, GRUPO_EDAD=18-29]
sup=0.111 conf=0.464 lift=1.201
[ENTIDAD=TAMAULIPAS, SEXO=HOMBRE]
→ [ESTATUS_VICTIMA=DESAPARECIDA, GRUPO_EDAD=18-29]
sup=0.091 conf=0.453 lift=1.172
[SEXO=MUJER]
→ [GRUPO_EDAD=18-29]
sup=0.073 conf=0.482 lift=1.165
[ESTATUS_VICTIMA=DESAPARECIDA, SEXO=MUJER]
→ [GRUPO_EDAD=18-29]
sup=0.071 conf=0.481 lift=1.163

[mlxtend Apriori] Ejecutando...

Tiempo: 0.02 s | Itemsets: 140 | Reglas: 230
Top-5 reglas (mlxtend Apriori) por lift:
[SEXO=MUJER]
→ [ESTATUS_VICTIMA=DESAPARECIDA, GRUPO_EDAD=18-29]
sup=0.071 conf=0.465 lift=1.204
[ENTIDAD=TAMAULIPAS]
→ [ESTATUS_VICTIMA=DESAPARECIDA, GRUPO_EDAD=18-29]
sup=0.111 conf=0.464 lift=1.201
[ENTIDAD=TAMAULIPAS, SEXO=HOMBRE]
→ [ESTATUS_VICTIMA=DESAPARECIDA, GRUPO_EDAD=18-29]
sup=0.091 conf=0.453 lift=1.172
[SEXO=MUJER]

```

→ [GRUPO_EDAD=18-29]
  sup=0.073  conf=0.482  lift=1.165
[ESTATUS_VICTIMA=DESAPARECIDA, SEXO=MUJER]
→ [GRUPO_EDAD=18-29]
  sup=0.071  conf=0.481  lift=1.163

[FP-Growth] Ejecutando...
  Tiempo: 0.04 s | Itemsets: 140 | Reglas: 230
Top-5 reglas (FP-Growth) por lift:
[SEXO=MUJER]
→ [ESTATUS_VICTIMA=DESAPARECIDA, GRUPO_EDAD=18-29]
  sup=0.071  conf=0.465  lift=1.204
[ENTIDAD=TAMAULIPAS]
→ [ESTATUS_VICTIMA=DESAPARECIDA, GRUPO_EDAD=18-29]
  sup=0.111  conf=0.464  lift=1.201
[ENTIDAD=TAMAULIPAS, SEXO=HOMBRE]
→ [ESTATUS_VICTIMA=DESAPARECIDA, GRUPO_EDAD=18-29]
  sup=0.091  conf=0.453  lift=1.172
[SEXO=MUJER]
→ [GRUPO_EDAD=18-29]
  sup=0.073  conf=0.482  lift=1.165
[ESTATUS_VICTIMA=DESAPARECIDA, SEXO=MUJER]
→ [GRUPO_EDAD=18-29]
  sup=0.071  conf=0.481  lift=1.163

Reglas interesantes (lift>1.2, conf>0.4): 2
[SEXO=MUJER] → [ESTATUS_VICTIMA=DESAPARECIDA, GRUPO_EDAD=18-29]
  sup=0.071  conf=0.465  lift=1.204
[ENTIDAD=TAMAULIPAS] → [ESTATUS_VICTIMA=DESAPARECIDA, GRUPO_EDAD=18-29]
  sup=0.111  conf=0.464  lift=1.201

Comparativa de tiempos (Calderon (2006-2012))
Apriori propio : 0.42 s
mlxtend Apriori : 0.02 s
FP-Growth      : 0.04 s

=====
SEXENIO: PeñaNieto (2012-2018) (11,751 registros)
=====

Transacciones usadas: 11,705
Parámetros → min_support=0.04, min_confidence=0.4

[Apriori propio] Ejecutando...

```

Tiempo: 0.67 s | Itemsets: 135 | Reglas: 183
 Top-5 reglas (Apriori propio) por lift:
 [GRUPO_EDAD=12-17]
 → [ESTATUS_VICTIMA=DESAPARECIDA, SEXO=MUJER]
 sup=0.051 conf=0.602 lift=3.131
 [ESTATUS_VICTIMA=DESAPARECIDA, GRUPO_EDAD=12-17]
 → [ENTIDAD=ESTADO DE MÉXICO]
 sup=0.042 conf=0.499 lift=3.088
 [GRUPO_EDAD=12-17]
 → [ENTIDAD=ESTADO DE MÉXICO, ESTATUS_VICTIMA=DESAPARECIDA]
 sup=0.042 conf=0.496 lift=3.070
 [GRUPO_EDAD=12-17]
 → [ENTIDAD=ESTADO DE MÉXICO]
 sup=0.042 conf=0.496 lift=3.067
 [ESTATUS_VICTIMA=DESAPARECIDA, GRUPO_EDAD=12-17]
 → [SEXO=MUJER]
 sup=0.051 conf=0.607 lift=3.044

[mlxtend Apriori] Ejecutando...

Tiempo: 0.03 s | Itemsets: 135 | Reglas: 183
 Top-5 reglas (mlxtend Apriori) por lift:
 [GRUPO_EDAD=12-17]
 → [ESTATUS_VICTIMA=DESAPARECIDA, SEXO=MUJER]
 sup=0.051 conf=0.602 lift=3.131
 [ESTATUS_VICTIMA=DESAPARECIDA, GRUPO_EDAD=12-17]
 → [ENTIDAD=ESTADO DE MÉXICO]
 sup=0.042 conf=0.499 lift=3.088
 [GRUPO_EDAD=12-17]
 → [ENTIDAD=ESTADO DE MÉXICO, ESTATUS_VICTIMA=DESAPARECIDA]
 sup=0.042 conf=0.496 lift=3.070
 [GRUPO_EDAD=12-17]
 → [ENTIDAD=ESTADO DE MÉXICO]
 sup=0.042 conf=0.496 lift=3.067
 [ESTATUS_VICTIMA=DESAPARECIDA, GRUPO_EDAD=12-17]
 → [SEXO=MUJER]
 sup=0.051 conf=0.607 lift=3.044

[FP-Growth] Ejecutando...

Tiempo: 0.06 s | Itemsets: 135 | Reglas: 183
 Top-5 reglas (FP-Growth) por lift:
 [GRUPO_EDAD=12-17]
 → [ESTATUS_VICTIMA=DESAPARECIDA, SEXO=MUJER]
 sup=0.051 conf=0.602 lift=3.131

```

[ESTATUS_VICTIMA=DESAPARECIDA, GRUPO_EDAD=12-17]
→ [ENTIDAD=ESTADO DE MÉXICO]
  sup=0.042  conf=0.499  lift=3.088
[GRUPO_EDAD=12-17]
→ [ENTIDAD=ESTADO DE MÉXICO, ESTATUS_VICTIMA=DESAPARECIDA]
  sup=0.042  conf=0.496  lift=3.070
[GRUPO_EDAD=12-17]
→ [ENTIDAD=ESTADO DE MÉXICO]
  sup=0.042  conf=0.496  lift=3.067
[ESTATUS_VICTIMA=DESAPARECIDA, GRUPO_EDAD=12-17]
→ [SEXO=MUJER]
  sup=0.051  conf=0.607  lift=3.044

```

Reglas interesantes (lift>1.2, conf>0.4): 13

```

[GRUPO_EDAD=12-17] → [ESTATUS_VICTIMA=DESAPARECIDA, SEXO=MUJER]
  sup=0.051  conf=0.602  lift=3.131
[ESTATUS_VICTIMA=DESAPARECIDA, GRUPO_EDAD=12-17] → [ENTIDAD=ESTADO DE MÉXICO]
  sup=0.042  conf=0.499  lift=3.088
[GRUPO_EDAD=12-17] → [ENTIDAD=ESTADO DE MÉXICO, ESTATUS_VICTIMA=DESAPARECIDA]
  sup=0.042  conf=0.496  lift=3.070
[GRUPO_EDAD=12-17] → [ENTIDAD=ESTADO DE MÉXICO]
  sup=0.042  conf=0.496  lift=3.067
[ESTATUS_VICTIMA=DESAPARECIDA, GRUPO_EDAD=12-17] → [SEXO=MUJER]
  sup=0.051  conf=0.607  lift=3.044

```

Comparativa de tiempos (PeñaNieto (2012-2018))

```

Apriori propio : 0.67 s
mlxtend Apriori : 0.03 s
FP-Growth      : 0.06 s

```

```

=====
SEXENIO: AML0 (2018-2024) (24,729 registros)
=====

```

```

Transacciones usadas: 24,632
Parámetros → min_support=0.04, min_confidence=0.4

```

[Apriori propio] Ejecutando...

Tiempo: 1.57 s | Itemsets: 125 | Reglas: 157

Top-5 reglas (Apriori propio) por lift:

```

[ESTATUS_VICTIMA=DESAPARECIDA, GRUPO_EDAD=12-17]
→ [SEXO=MUJER]
  sup=0.047  conf=0.542  lift=2.586
[GRUPO_EDAD=12-17]

```

```

→ [SEXO=MUJER]
  sup=0.051  conf=0.537  lift=2.562
[GRUPO_EDAD=12-17]
→ [ESTATUS_VICTIMA=DESAPARECIDA, SEXO=MUJER]
  sup=0.047  conf=0.492  lift=2.504
[ENTIDAD=MICHOACÁN, SEXO=HOMBRE]
→ [ESTATUS_VICTIMA=DESAPARECIDA, GRUPO_EDAD=30-59]
  sup=0.050  conf=0.522  lift=1.195
[ENTIDAD=SINALOA]
→ [ESTATUS_VICTIMA=DESAPARECIDA, SEXO=HOMBRE]
  sup=0.050  conf=0.874  lift=1.185

[mlxtend Apriori] Ejecutando...
Tiempo: 0.04 s | Itemsets: 125 | Reglas: 157
Top-5 reglas (mlxtend Apriori) por lift:
[ESTATUS_VICTIMA=DESAPARECIDA, GRUPO_EDAD=12-17]
→ [SEXO=MUJER]
  sup=0.047  conf=0.542  lift=2.586
[GRUPO_EDAD=12-17]
→ [SEXO=MUJER]
  sup=0.051  conf=0.537  lift=2.562
[GRUPO_EDAD=12-17]
→ [ESTATUS_VICTIMA=DESAPARECIDA, SEXO=MUJER]
  sup=0.047  conf=0.492  lift=2.504
[ENTIDAD=MICHOACÁN, SEXO=HOMBRE]
→ [ESTATUS_VICTIMA=DESAPARECIDA, GRUPO_EDAD=30-59]
  sup=0.050  conf=0.522  lift=1.195
[ENTIDAD=SINALOA]
→ [ESTATUS_VICTIMA=DESAPARECIDA, SEXO=HOMBRE]
  sup=0.050  conf=0.874  lift=1.185

[FP-Growth] Ejecutando...
Tiempo: 0.11 s | Itemsets: 125 | Reglas: 157
Top-5 reglas (FP-Growth) por lift:
[ESTATUS_VICTIMA=DESAPARECIDA, GRUPO_EDAD=12-17]
→ [SEXO=MUJER]
  sup=0.047  conf=0.542  lift=2.586
[GRUPO_EDAD=12-17]
→ [SEXO=MUJER]
  sup=0.051  conf=0.537  lift=2.562
[GRUPO_EDAD=12-17]
→ [ESTATUS_VICTIMA=DESAPARECIDA, SEXO=MUJER]
  sup=0.047  conf=0.492  lift=2.504

```

```
[ENTIDAD=MICHOACÁN, SEXO=HOMBRE]
→ [ESTATUS_VICTIMA=DESAPARECIDA, GRUPO_EDAD=30-59]
  sup=0.050  conf=0.522  lift=1.195
[ENTIDAD=SINALOA]
→ [ESTATUS_VICTIMA=DESAPARECIDA, SEXO=HOMBRE]
  sup=0.050  conf=0.874  lift=1.185
```

```
Reglas interesantes (lift>1.2, conf>0.4): 3
[ESTATUS_VICTIMA=DESAPARECIDA, GRUPO_EDAD=12-17] → [SEXO=MUJER]
  sup=0.047  conf=0.542  lift=2.586
[GRUPO_EDAD=12-17] → [SEXO=MUJER]
  sup=0.051  conf=0.537  lift=2.562
[GRUPO_EDAD=12-17] → [ESTATUS_VICTIMA=DESAPARECIDA, SEXO=MUJER]
  sup=0.047  conf=0.492  lift=2.504
```

```
Comparativa de tiempos (AMLO (2018-2024))
Apriori propio : 1.57 s
mlxtend Apriori : 0.04 s
FP-Growth      : 0.11 s
```

```
=====
SEXENIO: Sheinbaum (2024-) (6,515 registros)
=====
```

```
Transacciones usadas: 6,476
Parámetros → min_support=0.04, min_confidence=0.4
```

```
[Apriori propio] Ejecutando...
Tiempo: 0.40 s | Itemsets: 133 | Reglas: 191
Top-5 reglas (Apriori propio) por lift:
[GRUPO_EDAD=12-17]
→ [SEXO=MUJER]
  sup=0.082  conf=0.562  lift=2.218
[GRUPO_EDAD=12-17]
→ [ESTATUS_VICTIMA=DESAPARECIDA, SEXO=MUJER]
  sup=0.082  conf=0.562  lift=2.218
[ESTATUS_VICTIMA=DESAPARECIDA, GRUPO_EDAD=12-17]
→ [SEXO=MUJER]
  sup=0.082  conf=0.562  lift=2.218
[ENTIDAD=SINALOA]
→ [ESTATUS_VICTIMA=DESAPARECIDA, GRUPO_EDAD=30-59, SEXO=HOMBRE]
  sup=0.042  conf=0.441  lift=1.274
[ENTIDAD=SINALOA]
→ [GRUPO_EDAD=30-59, SEXO=HOMBRE]
```


sup=0.042 conf=0.441 lift=1.271

[mlxtend Apriori] Ejecutando...

Tiempo: 0.01 s | Itemsets: 133 | Reglas: 191

Top-5 reglas (mlxtend Apriori) por lift:

[GRUPO_EDAD=12-17]

→ [SEXO=MUJER]

sup=0.082 conf=0.562 lift=2.218

[GRUPO_EDAD=12-17]

→ [ESTATUS_VICTIMA=DESAPARECIDA, SEXO=MUJER]

sup=0.082 conf=0.562 lift=2.218

[ESTATUS_VICTIMA=DESAPARECIDA, GRUPO_EDAD=12-17]

→ [SEXO=MUJER]

sup=0.082 conf=0.562 lift=2.218

[ENTIDAD=SINALOA]

→ [ESTATUS_VICTIMA=DESAPARECIDA, GRUPO_EDAD=30-59, SEXO=HOMBRE]

sup=0.042 conf=0.441 lift=1.274

[ENTIDAD=SINALOA]

→ [GRUPO_EDAD=30-59, SEXO=HOMBRE]

sup=0.042 conf=0.441 lift=1.271

[FP-Growth] Ejecutando...

Tiempo: 0.03 s | Itemsets: 133 | Reglas: 191

Top-5 reglas (FP-Growth) por lift:

[GRUPO_EDAD=12-17]

→ [SEXO=MUJER]

sup=0.082 conf=0.562 lift=2.218

[GRUPO_EDAD=12-17]

→ [ESTATUS_VICTIMA=DESAPARECIDA, SEXO=MUJER]

sup=0.082 conf=0.562 lift=2.218

[ESTATUS_VICTIMA=DESAPARECIDA, GRUPO_EDAD=12-17]

→ [SEXO=MUJER]

sup=0.082 conf=0.562 lift=2.218

[ENTIDAD=SINALOA]

→ [ESTATUS_VICTIMA=DESAPARECIDA, GRUPO_EDAD=30-59, SEXO=HOMBRE]

sup=0.042 conf=0.441 lift=1.274

[ENTIDAD=SINALOA, ESTATUS_VICTIMA=DESAPARECIDA]

→ [GRUPO_EDAD=30-59, SEXO=HOMBRE]

sup=0.042 conf=0.441 lift=1.271

Reglas interesantes (lift>1.2, conf>0.4): 12

[GRUPO_EDAD=12-17] → [ESTATUS_VICTIMA=DESAPARECIDA, SEXO=MUJER]

sup=0.082 conf=0.562 lift=2.218

```

[GRUPO_EDAD=12-17] → [SEXO=MUJER]
  sup=0.082  conf=0.562  lift=2.218
[ESTATUS_VICTIMA=DESAPARECIDA, GRUPO_EDAD=12-17] → [SEXO=MUJER]
  sup=0.082  conf=0.562  lift=2.218
[ENTIDAD=SINALOA] → [ESTATUS_VICTIMA=DESAPARECIDA, GRUPO_EDAD=30-59, SEXO=HOMBRE]
  sup=0.042  conf=0.441  lift=1.274
[ENTIDAD=SINALOA, ESTATUS_VICTIMA=DESAPARECIDA] → [GRUPO_EDAD=30-59, SEXO=HOMBRE]
  sup=0.042  conf=0.441  lift=1.271

```

Comparativa de tiempos (Sheinbaum (2024-))

```

Apriori propio   : 0.40 s
mlxtend Apriori  : 0.01 s
FP-Growth        : 0.03 s

```

===== RESUMEN COMPARATIVO DE TIEMPOS POR SEXENIO =====

	sexenio	registros	t_propio	t_mlx	t_fp	reglas_propio	reglas_mlx	reg
Fox (2000-2006)		600	0.033150	0.008434	0.008185	185	185	
Calderon (2006-2012)		7039	0.421088	0.018313	0.035715	230	230	
PeñaNieto (2012-2018)		11705	0.670581	0.030452	0.055552	183	183	
AMLO (2018-2024)		24632	1.572022	0.039223	0.110566	157	157	
Sheinbaum (2024-)		6476	0.395483	0.012487	0.032943	191	191	

===== RESUMEN FINAL =====

Tiempos algoritmos Apriori:

```

Apriori propio   : 0.40 s  (O(2^n) candidatos, el mas lento)
mlxtend Apriori  : 0.01 s
FP-Growth        : 0.03 s  (sin generaci3n de candidatos, el m1s r1pido)

```

Algoritmos: Clasificación

En este caso se trató el estatus CONFIDENCIAL como eliminado, al igual que en la sección anterior.

Se construyen dos modelos de clasificación — **Árbol de Decisión** y **Random Forest** — para predecir la variable objetivo ESTATUS_VICTIMA a partir de las demás variables del dataset.

Objetivo: Predecir ESTATUS_VICTIMA (DESAPARECIDA / NO LOCALIZADA).

Preparación del dataset

Se parte del mismo dataset limpio usado en la sección anterior, aplicando las mismas transformaciones: eliminación de registros CONFIDENCIAL, discretización de edad en grupos de riesgo y clasificación del año de desaparición por sexenio.

```
import pandas as pd
import numpy as np

df = pd.read_csv("data_secretariado.csv")

# Eliminar registros con CONFIDENCIAL en columnas clave
cols_clave = ["SEXO", "ESTATUS_VICTIMA", "ENTIDAD",
              "FECHA_NACIMIENTO", "FECHA_DESAPARICION"]

for col in cols_clave:
    df = df[df[col] != "CONFIDENCIAL"]

df = df[df["SEXO"] != "INDETERMINADO"]

# Parsear fechas
df["FECHA_NACIMIENTO"] = pd.to_datetime(df["FECHA_NACIMIENTO"], errors="coerce")
df["FECHA_DESAPARICION"] = pd.to_datetime(df["FECHA_DESAPARICION"], errors="coerce")
df = df.dropna(subset=["FECHA_NACIMIENTO", "FECHA_DESAPARICION"])
```

```

# Calcular edad al momento de la desaparición
df["EDAD"] = ((df["FECHA_DESAPARICION"] - df["FECHA_NACIMIENTO"])
               .dt.days // 365).astype(int)
df = df[df["EDAD"].between(0, 120)]

# Discretizar edad en grupos de riesgo
bins = [-1, 11, 17, 29, 59, 120]
labels = ["0-11", "12-17", "18-29", "30-59", "60+"]
df["GRUPO_EDAD"] = pd.cut(df["EDAD"], bins=bins, labels=labels)

# Extraer mes y sexenio de la fecha de desaparición
meses = {1:"Enero", 2:"Febrero", 3:"Marzo", 4:"Abril", 5:"Mayo",
          6:"Junio", 7:"Julio", 8:"Agosto", 9:"Septiembre",
          10:"Octubre", 11:"Noviembre", 12:"Diciembre"}
df["MES_DESAPARICION"] = df["FECHA_DESAPARICION"].dt.month.map(meses)

anio = df["FECHA_DESAPARICION"].dt.year
df["SEXENIO"] = pd.cut(
    anio,
    bins=[1999, 2006, 2012, 2018, 2024, 2030],
    labels=["Fox", "Calderón", "Peña Nieto", "AMLO", "Sheinbaum"]
)

# Quedarse solo con las dos clases objetivo
df = df[df["ESTATUS_VICTIMA"].isin(["DESAPARECIDA", "NO LOCALIZADA"])]

print(f"Registros tras limpieza: {len(df):,}")
print()
print(df["ESTATUS_VICTIMA"].value_counts())

```

Registros tras limpieza: 50,867

```

ESTATUS_VICTIMA
DESAPARECIDA      48116
NO LOCALIZADA     2751
Name: count, dtype: int64

```

Consideraciones sobre el desbalance de clases

```

conteo = df["ESTATUS_VICTIMA"].value_counts()
pct     = df["ESTATUS_VICTIMA"].value_counts(normalize=True) * 100

print("Distribución de ESTATUS_VICTIMA:")
for clase in conteo.index:
    print(f" {clase:<20} {conteo[clase]:>7,} ({pct[clase]:.1f}%)")

```

```

Distribución de ESTATUS_VICTIMA:
DESAPARECIDA          48,116 (94.6%)
NO LOCALIZADA         2,751 (5.4%)

```

El dataset presenta un **fuerte desbalance**: la clase DESAPARECIDA representa aproximadamente el 94% de los registros válidos. Un modelo que predijera siempre DESAPARECIDA tendría una accuracy altísima sin aprender nada útil. Por tal razón se optó por no usar este modelo.

Selección de métricas y justificación

Table 1: Métricas seleccionadas y justificación

Métrica	Fórmula	Por qué se usa
F1-score (macro)	$\frac{2 \cdot \text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}$	Balancea precisión y recall para ambas clases por igual, penalizando el desempeño pobre en la clase minoritaria (NO LOCALIZADA). Es la métrica principal.
Recall por clase	$\frac{TP}{TP + FN}$	En desapariciones es crítico no pasar por alto a una víctima NO LOCALIZADA. Recall alto en esa clase significa que el modelo identifica correctamente esos casos.
Precision por clase	$\frac{TP}{TP + FP}$	Indica cuántas predicciones positivas son correctas, evitando falsas alarmas.
Accuracy	$\frac{TP + TN}{Total}$	Como mencionamos anteriormente esta métrica no se usará por el desvalance.

Relación con el mundo real: En búsqueda de personas, un falso negativo — clasificar como DESAPARECIDA a alguien NO LOCALIZADA — puede implicar que el caso no reciba la atención urgente que requiere. Por eso el recall en NO LOCALIZADA tiene mayor peso práctico que la accuracy global.

Codificación de variables y división train/test

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder

# Features y variable objetivo
features = ["SEXO", "GRUPO_EDAD", "ENTIDAD", "MES_DESAPARICION", "SEXENIO"]
target    = "ESTATUS_VICTIMA"

df_model = df[features + [target]].dropna()

# Codificación de variables categóricas
df_enc = df_model.copy()
encoders = {}
for col in features:
    le = LabelEncoder()
    df_enc[col] = le.fit_transform(df_enc[col].astype(str))
    encoders[col] = le

le_target = LabelEncoder()
df_enc[target] = le_target.fit_transform(df_enc[target])

X = df_enc[features]
y = df_enc[target]

# División 80/20 estratificada
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print(f"Entrenamiento : {len(X_train):,} registros")
print(f"Prueba          : {len(X_test):,} registros")
print(f"Features          : {features}")
print(f"Clases             : {le_target.classes_}")
```

```
Entrenamiento : 40,451 registros
Prueba        : 10,113 registros
Features      : ['SEXO', 'GRUPO_EDAD', 'ENTIDAD', 'MES_DESAPARICION', 'SEXENIO']
Clases        : ['DESAPARECIDA' 'NO LOCALIZADA']
```

Se usa una división **80/20 estratificada** (stratify=y) para garantizar que la proporción

entre clases se mantenga igual en entrenamiento y prueba, lo cual es esencial con datasets desbalanceados.

Entrenamiento de los modelos

Árbol de Decisión

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import classification_report, accuracy_score

dt = DecisionTreeClassifier(
    max_depth=8,          # limita profundidad para evitar sobreajuste
    min_samples_leaf=50,  # cada hoja requiere mínimo 50 muestras
    class_weight="balanced", # compensa el desbalance de clases
    random_state=42
)
dt.fit(X_train, y_train)
y_pred_dt = dt.predict(X_test)

print("  Árbol de Decisión  ")
print(f"Accuracy : {accuracy_score(y_test, y_pred_dt):.4f}\n")
print(classification_report(y_test, y_pred_dt,
                           target_names=le_target.classes_))
```

Árbol de Decisión

Accuracy : 0.7792

	precision	recall	f1-score	support
DESAPARECIDA	0.99	0.78	0.87	9565
NO LOCALIZADA	0.17	0.82	0.29	548
accuracy			0.78	10113
macro avg	0.58	0.80	0.58	10113
weighted avg	0.94	0.78	0.84	10113

Random Forest

```

from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(
    n_estimators=100,          # 100 árboles en el bosque
    max_depth=10,
    min_samples_leaf=30,
    class_weight="balanced",  # compensa el desbalance de clases
    n_jobs=-1,                # usa todos los núcleos disponibles
    random_state=42
)
rf.fit(X_train, y_train)
y_pred_rf = rf.predict(X_test)

print("  Random Forest  ")
print(f"Accuracy : {accuracy_score(y_test, y_pred_rf):.4f}\n")
print(classification_report(y_test, y_pred_rf,
                             target_names=le_target.classes_))

```

Random Forest
Accuracy : 0.8115

	precision	recall	f1-score	support
DESAPARECIDA	0.98	0.81	0.89	9565
NO LOCALIZADA	0.19	0.78	0.31	548
accuracy			0.81	10113
macro avg	0.59	0.80	0.60	10113
weighted avg	0.94	0.81	0.86	10113

Comparativa entre modelos

```

from sklearn.metrics import f1_score, precision_score, recall_score

modelos = {
    "Árbol de Decisión": y_pred_dt,
    "Random Forest":     y_pred_rf,
}

```



```

idx_no_loc = list(le_target.classes_).index("NO LOCALIZADA")

print(f"{'Modelo':<22} {'Accuracy':>10} {'F1 macro':>10} "
      f"{'Recall NO LOC':>15} {'Prec NO LOC':>13}")
print(" " * 74)
for nombre, y_pred in modelos.items():
    acc = accuracy_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred, average="macro")
    rec = recall_score(y_test, y_pred, average=None)[idx_no_loc]
    prec = precision_score(y_test, y_pred, average=None)[idx_no_loc]
    print(f"{nombre:<22} {acc:>10.4f} {f1:>10.4f} {rec:>15.4f} {prec:>13.4f}")

```

Modelo	Accuracy	F1 macro	Recall NO LOC	Prec NO LOC
Árbol de Decisión	0.7792	0.5777	0.8157	0.1733
Random Forest	0.8115	0.6004	0.7810	0.1933

Variables más relevantes

```

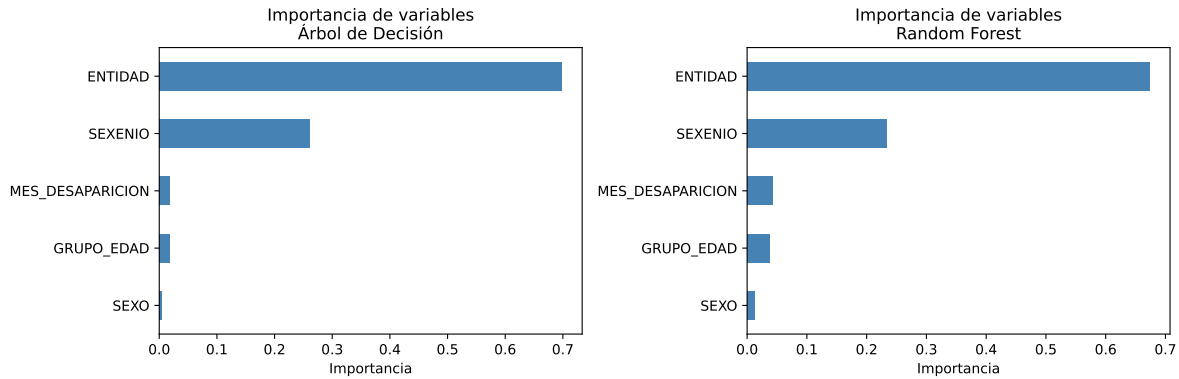
import matplotlib.pyplot as plt

fig, axes = plt.subplots(1, 2, figsize=(12, 4))

for ax, (nombre, modelo) in zip(axes, [("Árbol de Decisión", dt),
                                       ("Random Forest", rf)]):
    imp = pd.Series(modelo.feature_importances_, index=features)
    imp.sort_values().plot(kind="barh", ax=ax, color="steelblue")
    ax.set_title(f"Importancia de variables\n{n{nombre}}")
    ax.set_xlabel("Importancia")

plt.tight_layout()
#plt.savefig("images/importancia_variables.png", dpi=150, bbox_inches="tight")
plt.show()

```



Ambos modelos coinciden en que **ENTIDAD**, **GRUPO_EDAD** y **SEXO** son las variables con mayor poder predictivo, lo cual es consistente con los patrones encontrados en la sección de reglas de asociación.

Discusión: ¿patrones reales o sesgos?

Los modelos capturan patrones **estadísticamente reales** dentro del dataset disponible, con lo anterior podemos decir que:

- **Sesgo de registro:** El 37% de los registros originales son **CONFIDENCIAL** y fueron excluidos. Si la confidencialidad no es aleatoria, el modelo aprende patrones sesgados hacia las entidades con mayor transparencia en el registro.
- **Desbalance de clases:** Aunque se usó `class_weight="balanced"` para compensarlo, el modelo sigue teniendo más información sobre **DESAPARECIDA** que sobre **NO LOCALIZADA**. El recall en la clase minoritaria refleja este límite estructural.
- **Cobertura temporal desigual:** Los sexenios con menos registros (Fox, Calderón) tienen menor representación en el entrenamiento, por lo que el modelo puede no generalizar bien a patrones históricos.
- **Variables ausentes:** El dataset no incluye variables socioeconómicas, contexto de violencia regional ni tipo de desaparición, que podrían ser predictoras importantes del estatus final de la víctima.

En conclusión, los modelos reflejan en parte el fenómeno real y en parte las limitaciones del sistema de registro. Deben usarse como herramienta exploratoria, no como sistema de decisión definitivo.